



Web Service Paradigms & Implementation

The REST Architectural Style



Content

Chapter 3: The REST Architectural Style

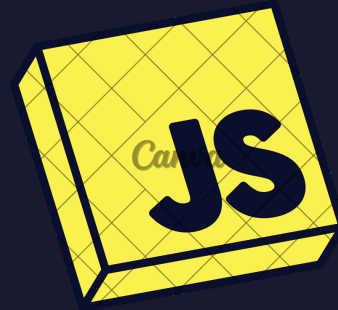
- 3.1 What is REST? (Representational State Transfer)
 - Roy Fielding's Doctoral Dissertation.
 - REST as an architectural style, not a protocol.
- 3.2 The Six Constraints of REST
 - Client-Server, Stateless, Cacheable, Uniform Interface, Layered System, Code-on-Demand.
- 3.3 RESTful Principles in Practice
 - Resources and URIs (Noun-based, not verbs).
 - HTTP Methods as Actions (CRUD operations).
 - Representations (JSON, XML).
- 3.4 Designing a RESTful API



In this course we are going to build a simple CRUD REST API app using **Node.js**, **JavaScript**, & **Express**.
We will build REST API containing a list of items.
We will be able to **create**, **read**, **update**, & **delete** items in our list via our REST API.

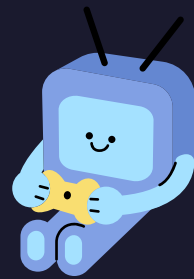
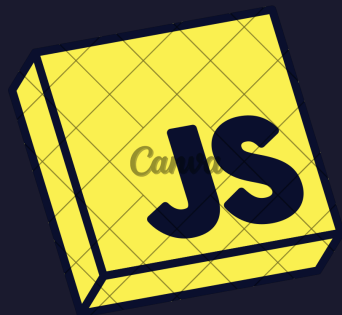


Exploring JavaScript, Node.js, & Express



Exploring JavaScript

JavaScript is a scripting language enabling you to create dynamic content, control multimedia, animate images, & much more across web, desktop, and mobile platforms



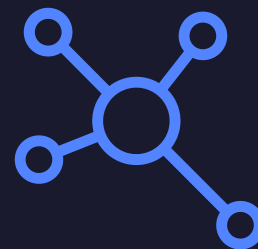
Object-Centered Script Language



Client-edge technology



Interpreted language



Ability to perform In Built Function



Ability to handle events



Exploring Node.js

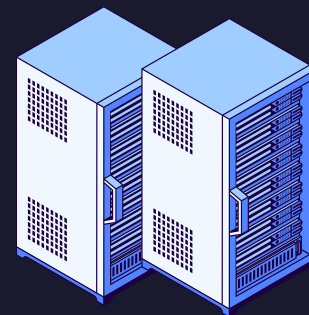
Node.js is an open-source JavaScript runtime environment that is cross-platform, asynchronous, & event-driven used to build fast & scalable server-side apps



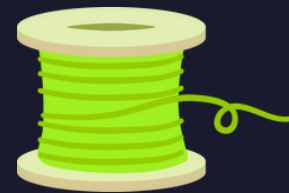
Open Source



Asynchronous (libuv library)



Scalability



Single threaded (Because of Javascript)



Cross platform



Exploring Express

Express.js is a lightweight & flexible open source framework for Node.js supporting web or mobile development. It manages routing, sessions, HTTP/S requests, error handling & more.

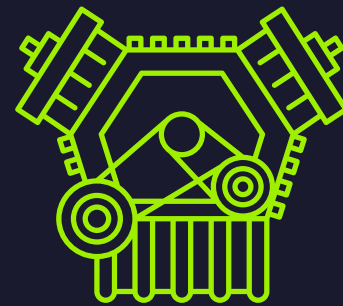
Express



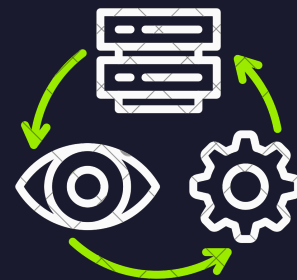
HTTP utility methods & middleware to ease building API's



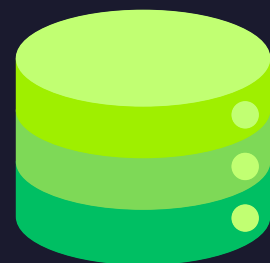
Faster web development



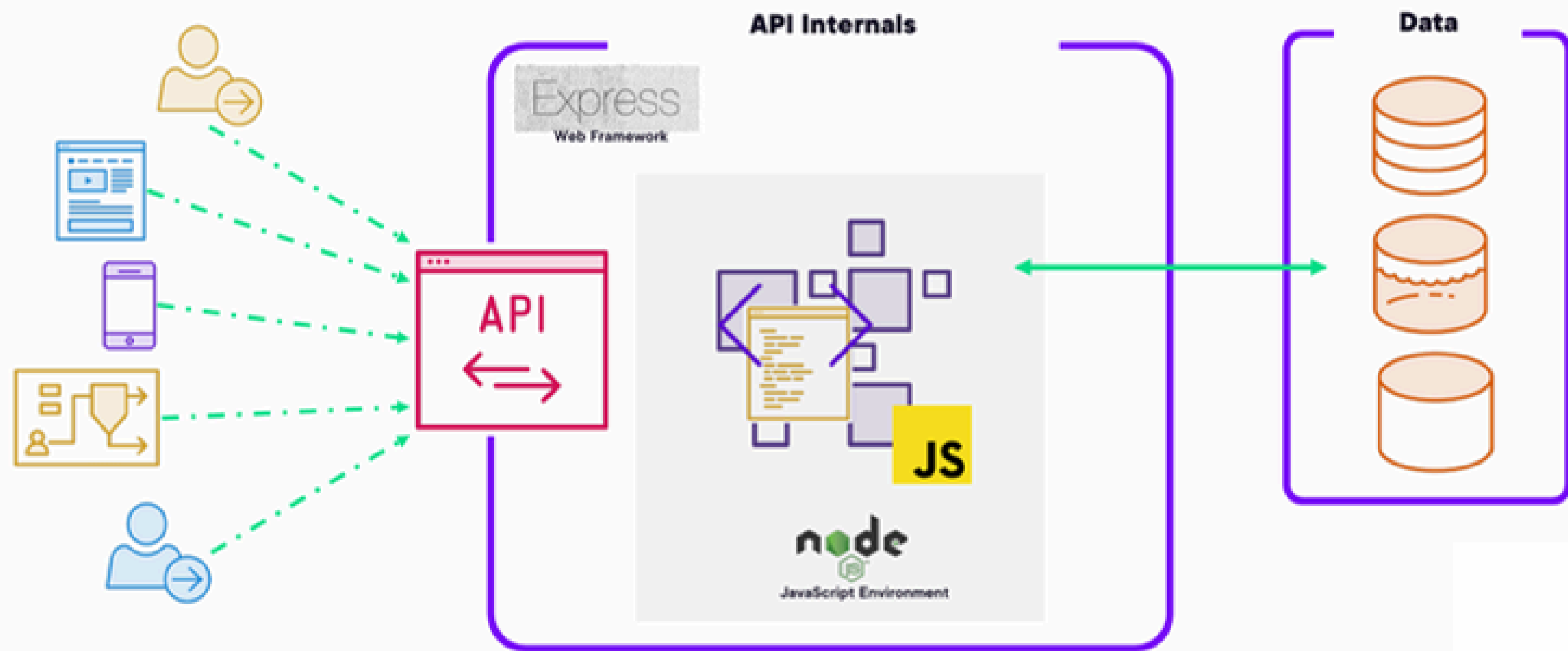
Provides templating engines: Jade & EJS



Model-View-Controller (MVC) architecture based



Integration with databases such as MongoDB, Redis, MySQL





Exploring the REST API

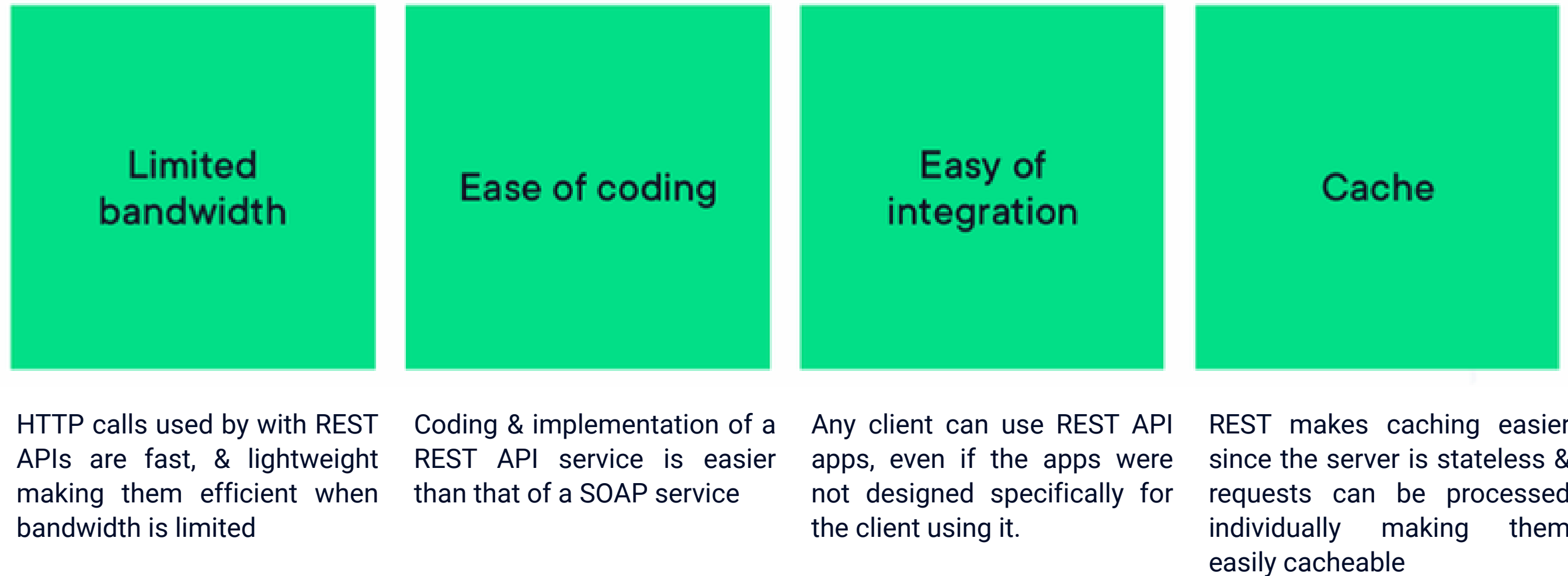
Exploring REST API

REST or RESTful is the internet's language it stands for **RE**presentational **S**tate **T**ransfer.

REST is an architectural style & an approach for communications often used in web dev & for systems to communicate.

REST API is an interface used by two or more computer systems to exchange data securely via HTTP requests GET, PUT, POST & DELETE over the internet.

Reasons for REST APIs



REST API Concepts

Client

- software that runs on a user's computer or smartphone and initiates communication

Server

- offers a REST API as a means of access to the app data or features

Stateless

- Client sends all info necessary to understand the request. The server does not have any stored context or session state & can not reuse info from any previous requests

Header

- store information relevant to both the client & server such as API key, name, IP address, & info about the response format

Body

- used to convey additional information to the server such as data you want to add or replace

Endpoint

- a Uniform Resource Identifier (URI) indicating where and how to find the resource on the Internet such as a Unique Resource Location (URL)

Cacheable

- require that the data within a response to a request be implicitly or explicitly labeled as cacheable or non-cacheable

Uniform Interface

- a consistent interface across all APIs that functions as a contract between the client & the service, & it is shared by all REST APIs

Layered System

- Every REST-enabled component has no access to components other than the one with whom it is communicating

Code On Demand (CoD)

- allows client functionality to be extended by downloading and executing code in the form of applets or scripts

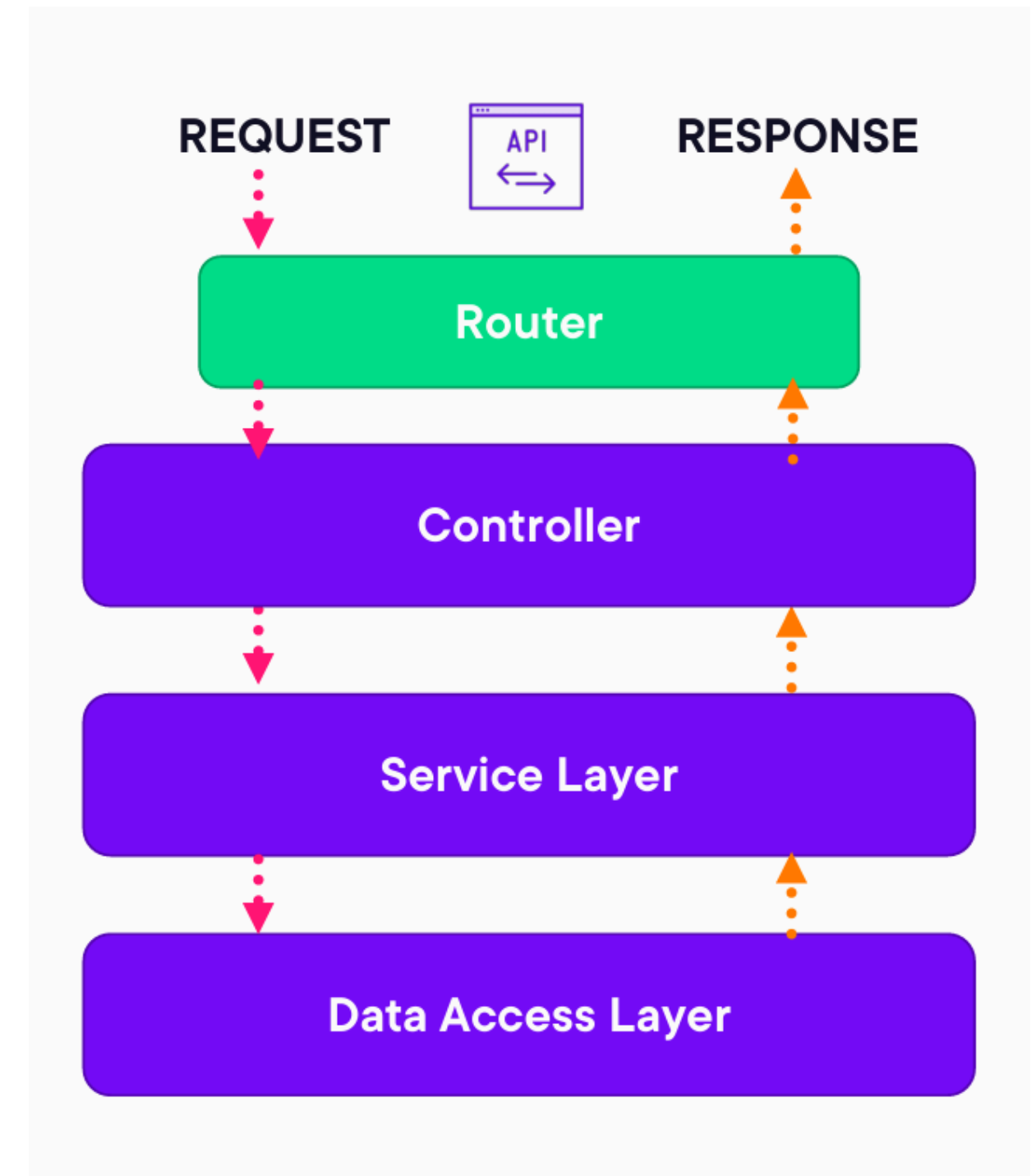
REST API Architecture

Router: from Express that passes requests to the corresponding controller

Controller: handles all stuff related to HTTP/S

Service Layer: has the business logic & exports services (methods) used by the controller

Data Access Layer: where interacting with the Database happens



REST API Benefits



Independent-client and server are independent. In other words, the REST protocol separates the data storage and the UI from the server



Scalability-REST APIs can be scaled by a dev team without much difficulty due to the separation between the client and the server.

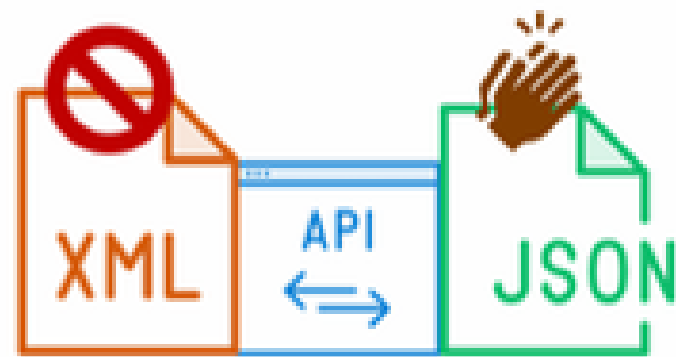


Easy Integration the use of ubiquitous standards, coupled with support across many languages, the use of HTTP protocol, & use by many devs with their apps are a recipe for ease of integration among many platforms.



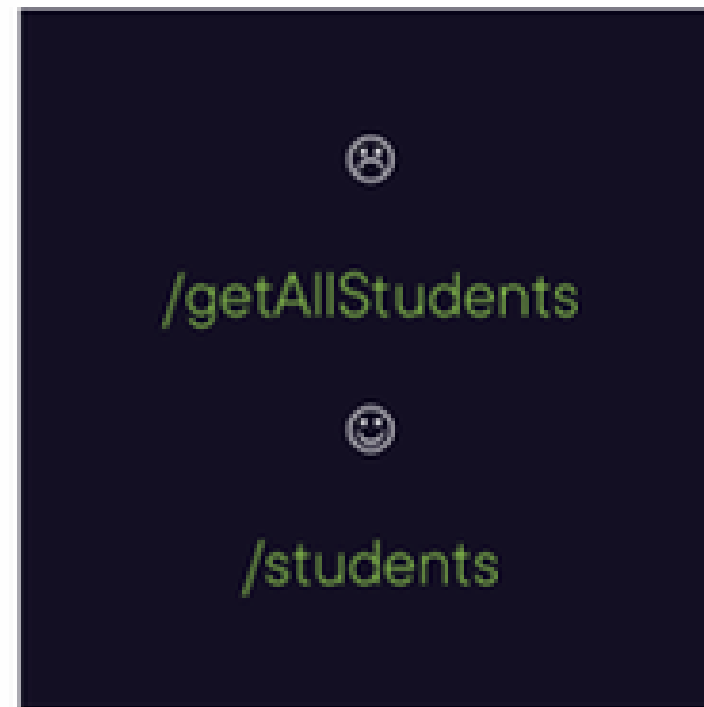
Flexibility-it is easy for devs to understand & Implement, it is format-agnostic with the ability to work with XML, JSON, HTML, etc., & easy to modify.

REST API Design Principles



Accept & respond with JSON

JavaScript has built-in methods to encode & decode JSON. Almost every networked



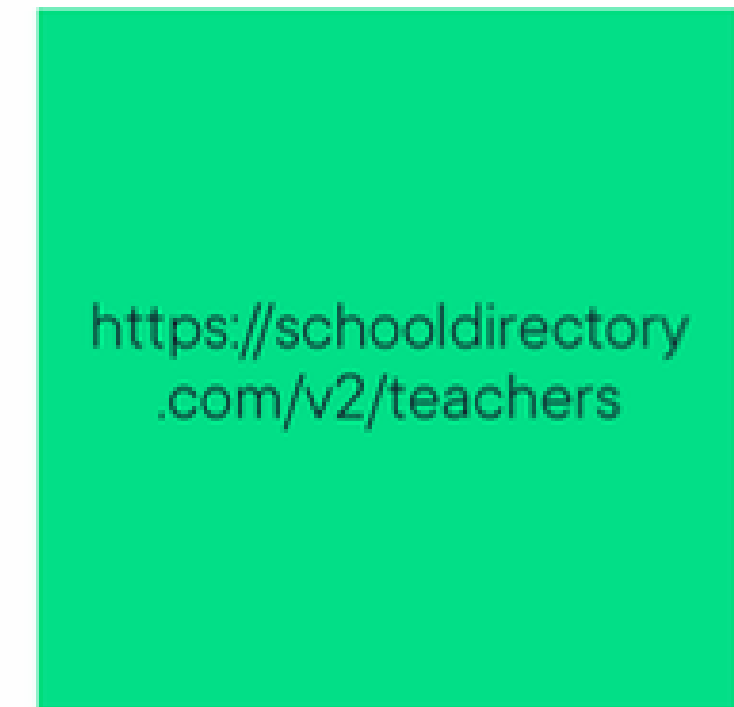
Use nouns instead of verbs in endpoint paths

Our HTTP request method already has the verb. It is better to let the HTTP



Allow API to filter, order, & paginate data

Limit the amount of data the REST API will return. This helps conserve bandwidth & the potential of overloading the REST API server.



Version the REST API

Versioning indicates a REST API's features & exposed resources. Versioning is typically done using `/v1/`, `/v2/`, etc. at the start of the REST API's path.

REST API Design Principles

```
const express =  
require('express');  
  
const bodyParser =  
require('body-parser');  
  
const redis = require('redis');  
  
const app = express();  
  
let cache = redisClient;  
  
app.use(cache("5 minutes"));
```

Use Caching

Caching data will improve performance

We can add caching to return data from the local memory cache instead of querying the database to get the data every time middleware such as `redis`, `apicache` etc is needed to handle the caching

Some Frequently used error codes in REST APIs:

200-Ok HTTP response for every success for GET, PUT or POST.

201-Created HTTP response for success and resource had created and not response body or response body is empty.

404-Not Found HTTP response for the requested resource is not available to access.

500 INTERNAL SERVER ERROR — HTTP response indicates that there might be a system failure.

```
app.post('/students', (req, res) => {  
  
  const { email } = req.body;  
  
  const userExists = users.find(st =>  
    st.email === email);  
  
  if (userExists) {  
  
    return res.status(404).json({ error:  
      'Student already exists' })  
  
  }  
})
```

Proper HTTP error handling

REST APIs use HTTP calls if an error/exception is thrown from the server side, it will give HTTP error. The REST API should handle errors gracefully & return HTTP response codes to the invoked client



Document

Document your API. Start documenting your API from its design phase. You can use a tool to help build your API documentation. Swagger is a set of open-source tools built around the OpenAPI Spec that can help build the documentation of your API.

Authentication Methods with a REST API

It is recommended to secure REST APIs access using SSL & TLS

There are four fundamental approaches to authentication for REST APIs

The easiest way to handle authentication with a REST API in JavaScript is to use HTTP basic auth. It sends a username & password alongside every API call.

This creates unique keys for devs & passes them alongside every request. The API generates a secret key that is a long, difficult-to-guess string of letters & numbers.

This orchestrates approvals automatically between an API owner & the service.

You can also authorize devs to obtain access on their own.

Another approach is to have no auth. Requests can be made to a specific URL & get a response without an API key or any credentials.

This is not recommended but is sometimes used for internal only APIs.



Practical part



End. CVE-1803-96. Thank you.